

Chapter 13

Kubernetes in Action 1st Edition

Securing Cluster Nodes and the Network

노드와 네트워크 보호 — Pod이 호스트와 옆 Pod에 닿는 길을 끊는다



신재근 (Jagger)

✓ 실측 검증됨 · minikube v1.38.1 + Calico CNI · K8s v1.35.1 · 2026-05-29

왜 13장인가

12장에서 우리가 담은 것

- API Server 북쪽 경계: "누가 API를 호출할 수 있는가"
- 인증(Token / Cert) → 인가(RBAC) → 어드미션
- ServiceAccount + Role/ClusterRoleBinding으로 권한 최소화

12장이 담지 못한 것

- API 호출 권한이 없어도 Pod 자체가 노드와 네트워크에 닿을 수 있다면?
- 한 Pod이 호스트 PID/네트워크 namespace에 들어가면 → kubelet, etcd 클라이언트 인증서, 다른 컨테이너 보임
- Pod이 root + privileged면 → 컨테이너 escape → 노드 탈취
- Pod이 다른 ns의 DB Pod에 직접 TCP 접속할 수 있다면 → API 우회 데이터 유출

13장이 채우는 4축

호스트 namespace 분리 / Security Context / Pod Security Admission / NetworkPolicy

사고 사례 — 13장이 막을 수 있었던 것

Capital One (2019, 1억 명 노출)

- WAF Pod의 IAM 메타데이터 엔드포인트(169.254.169.254) 접근 허용
- WAF가 SSRF로 메타데이터 서버 호출 → 임시 자격 증명 탈취 → S3 버킷 1억 명
- NetworkPolicy egress 차단으로 메타데이터 접근만 막아도 차단 가능했던 사고

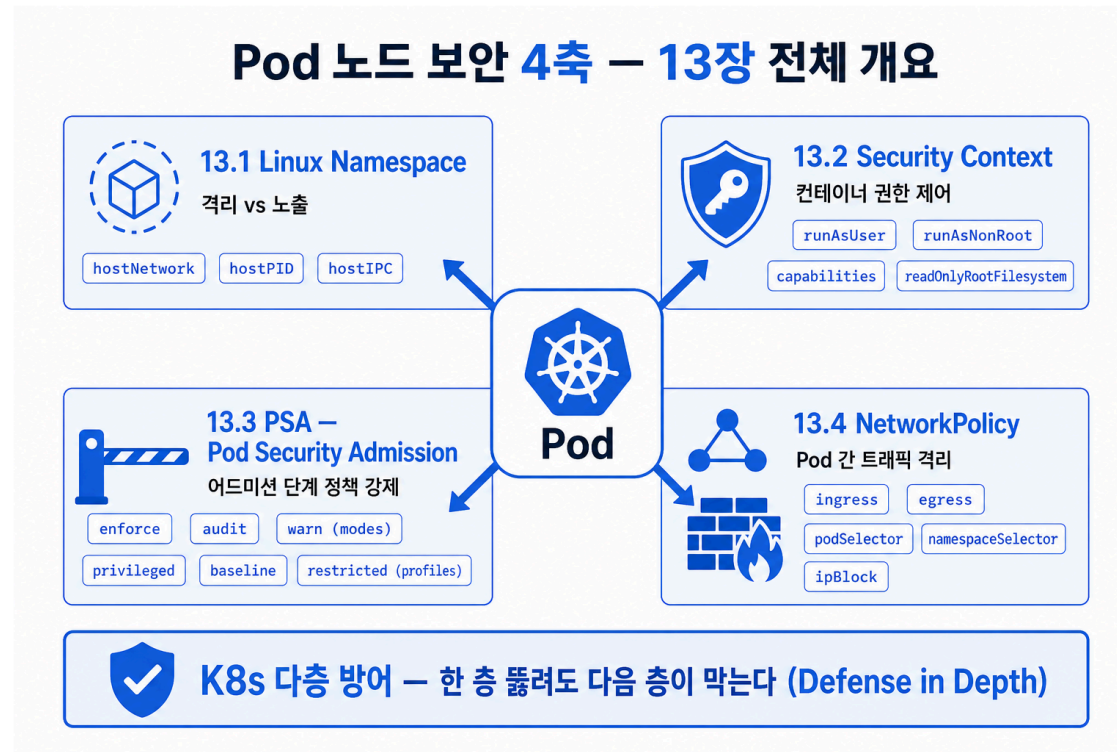
Argo CD CVE-2022-24348 (path traversal → 클러스터 탈취)

- Argo CD Pod의 default ServiceAccount + cluster-admin 권한
- path traversal로 임의 파일 읽기 → kubelet 인증서 → 노드 RCE
- Pod Security restricted + NetworkPolicy로 API Server 외 트래픽 차단으로 영향 축소

Tesla 2018 — Kubernetes 대시보드 노출

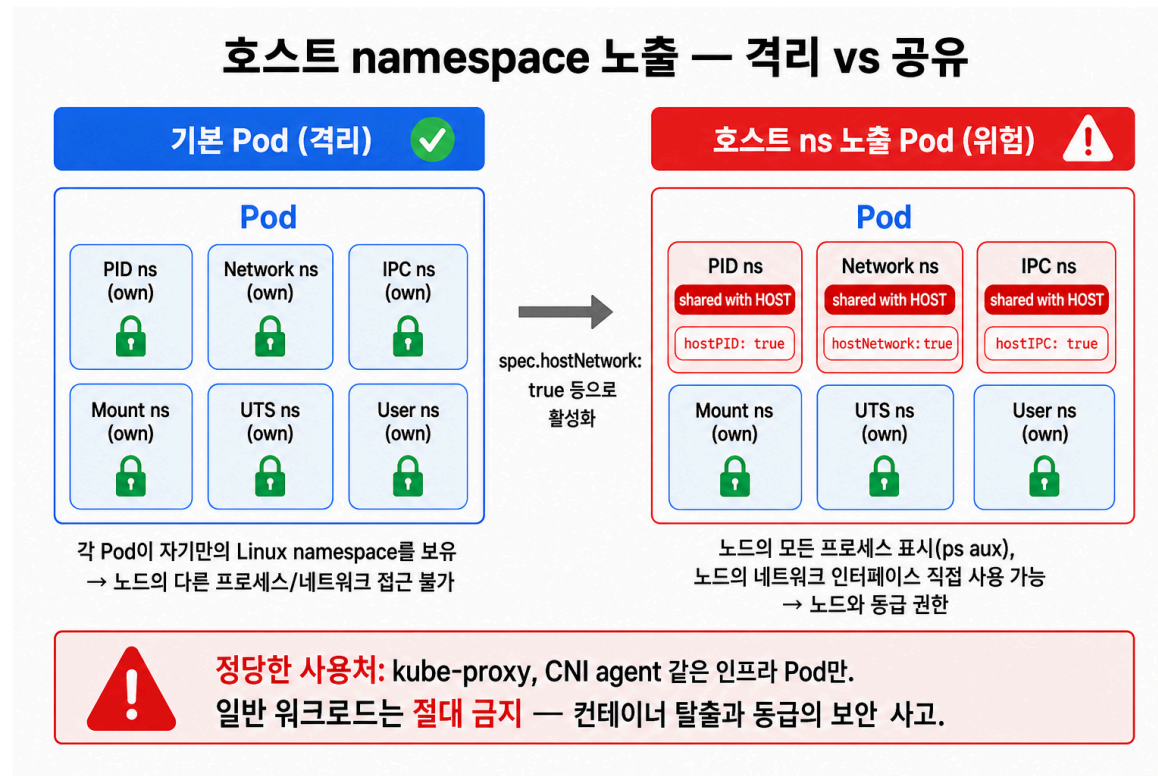
- 대시보드 Pod의 hostNetwork=true → 노드 IP로 인터넷 노출 → 비밀번호 없이 접근 → 크립토 채굴

13장의 4축 — 한 장 요약



4축이 동작하는 레벨이 모두 다름 — Pod spec(호스트 ns), Pod/Container spec(Security Context), namespace label(PSA), CNI 데이터플레인(NetworkPolicy). 겹쳐 적용해야 의미가 있는 Defense in Depth.

13.1 호스트 namespace — Pod이 노드를 들여다보는 창



3개 플래그 모두 false가 기본 — `hostNetwork` (노드 NIC/메타데이터), `hostPID` (노드 프로세스/kubelet 인자), `hostIPC` (SysV IPC). 시스템 Pod(CNI/kube-proxy)만 허용.

Demo 1 — 호스트 namespace 노출의 실체

```
# host-namespace-pod.yaml
apiVersion: v1
kind: Pod
metadata: { name: host-namespace-pod }
spec:
  hostNetwork: true    # 노드 NIC 보임
  hostPID: true        # 노드 프로세스 보임
  hostIPC: true        # 노드 IPC 보임
  containers:
  - { name: main, image: busybox:stable, command: ["sleep","3600"] }
```

```
kubectl apply -f host-namespace-pod.yaml
kubectl exec host-namespace-pod -- ps aux | head -8
kubectl exec host-namespace-pod -- ip addr | head -12
```

목적: Pod 안에서 노드의 systemd, kubelet, containerd, 그리고 노드 NIC가 보이는 것을 직접 눈으로 확인합니다.

Demo 1 — 실측 출력

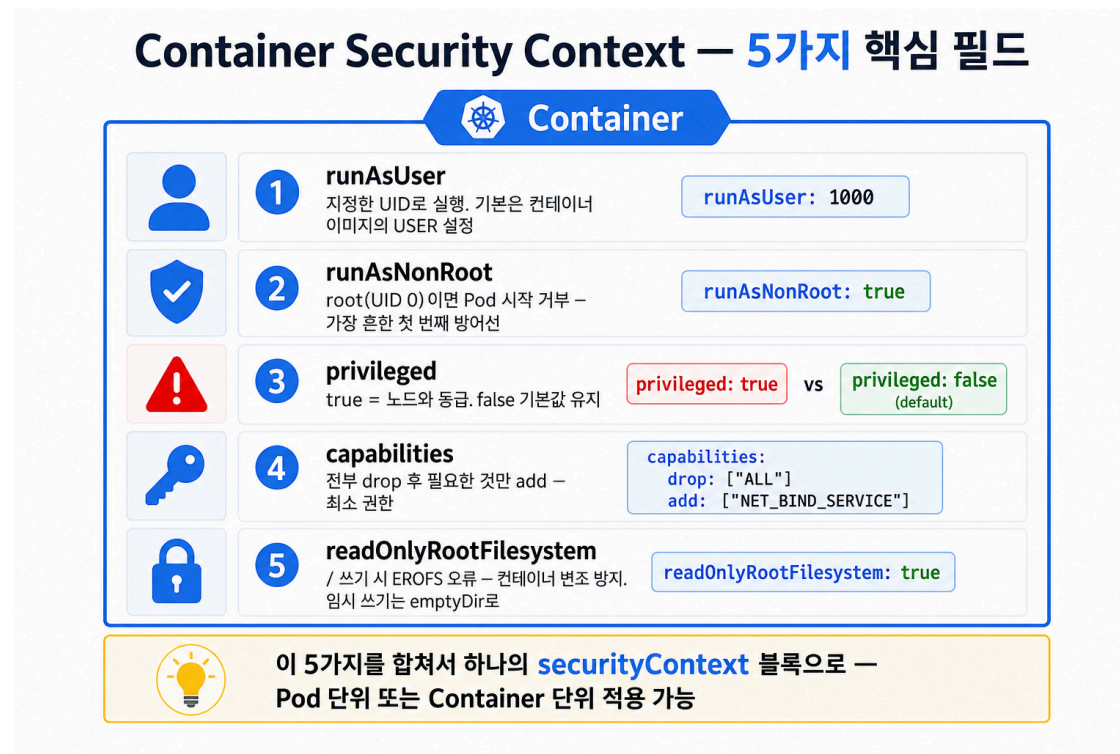
```
$ kubectl exec host-namespace-pod -- ps aux | head -8
PID    USER      TIME  COMMAND
   1   root         0:00 {systemd} /sbin/init
  94   root         0:00 /lib/systemd/systemd-journald
 109   root         5:10 /var/lib/minikube/binaries/v1.35.1/kubelet ... --hostname-override=minikube-m04 ...
 146   root         0:00 sshd: /usr/sbin/sshd -D [listener]
 506   root         2:44 /usr/bin/containerd
 680   root         0:05 /usr/bin/containerd-shim-runc-v2 -id 31c48be8...
```

```
$ kubectl exec host-namespace-pod -- ip addr | head -12
1: lo: <LOOPBACK,UP> ... inet 127.0.0.1/8
2: tunl0@NONE: <NOARP,UP> ... inet 172.21.59.128/32          # ← Calico tunl0 (노드 IP)
3: eth0@if8: <BROADCAST,MULTICAST,UP> ... fa:a9:b3:4e:32:56
```

- `systemd /sbin/init` — PID 1이 노드의 init 프로세스, Pod의 init이 아님
- `kubelet --hostname-override=minikube-m04` — 노드 kubelet의 명령행 인자가 그대로 노출
- `tunl0 172.21.59.128` — Pod IP가 아닌 노드 자신의 tunneling 인터페이스 IP

→ 이 Pod이 root였다면 `/proc/1/environ` 으로 노드 시크릿, kubelet config 모두 접근 가능합니다.

13.2 Security Context — 컨테이너 단위 권한의 5개 핵심 필드



PSA restricted 통과 5개 필드: `runAsNonRoot:true` + `runAsUser:non-zero` + `allowPrivilegeEscalation:false` + `capabilities.drop: ["ALL"]` + `seccompProfile:RuntimeDefault`. (권장) `readOnlyRootFilesystem:true` 추가. Helm values 기본 템플릿.

privileged — 절대 켜지 않는 단 하나

privileged: true가 켜지면

- 노드의 모든 디바이스(/dev) 접근
- 모든 Linux capability 자동 부여 (CAP_SYS_ADMIN 포함)
- AppArmor / seccomp 프로파일 무시
- Docker-in-Docker, GPU 드라이버 같은 특수 워크로드를 위해 만들어진 "탈출구"

컨테이너 escape의 표준 경로

```
privileged Pod → mount /dev/sda1 /host → chroot /host  
→ 노드 root 셸 → kubelet certificates 탈취 → 전체 클러스터
```

운영 원칙

- `privileged: true` 가 필요한 정상 워크로드는 사실상 없다
- CSI 드라이버, GPU operator 같은 시스템 Pod만 namespace 분리해서 허용
- PSA `baseline` / `restricted` 프로파일은 자동으로 차단
- 감사 쿼리: `kubectl get pods -A -o json | jq '.items[].spec.containers[]?.securityContext.privileged'`

Demo 2 — runAsUser + runAsNonRoot

```
# security-context-runas.yaml
apiVersion: v1
kind: Pod
metadata: { name: runas-pod }
spec:
  securityContext:          # Pod 레벨 (모든 컨테이너에 상속)
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
    runAsNonRoot: true
  containers:
  - { name: main, image: busybox:stable, command: ["sh","-c","id; sleep 3600"] }
```

```
$ kubectl exec runas-pod -- id
uid=1000 gid=3000 groups=2000,3000
```

- `uid=1000`, `gid=3000` — Pod spec 값과 정확히 일치, `groups=2000,3000` 은 fsGroup 추가됨
- `runAsNonRoot: true` — image USER가 root면 기동 거부 (PSA restricted 통과와 핵심)

Demo 3 — capabilities drop ALL + add NET_BIND_SERVICE

```
# security-context-caps.yaml
spec:
  containers:
  - name: main
    image: busybox:stable
    command: ["sh", "-c", "grep ^Cap /proc/1/status; sleep 3600"]
    securityContext:
      capabilities:
        drop: ["ALL"]
        add: ["NET_BIND_SERVICE"]    # 1024 미만 포트 바인딩만 허용
```

```
$ kubectl exec caps-pod -- sh -c 'grep ^Cap /proc/1/status'
CapInh: 0000000000000000
CapPrm: 0000000000000400    # ← 0x400 = bit 10 = CAP_NET_BIND_SERVICE
CapEff: 0000000000000400
CapBnd: 0000000000000400
CapAmb: 0000000000000000
```

- 기본 컨테이너는 14개 capability(약 `0x00000000a80425fb`) — 그 중 다수가 위험(SYS_ADMIN, NET_RAW, SYS_PTRACE 등)
- `**drop ["ALL"] + add ["NET_BIND_SERVICE"]**`로 정확히 1개 비트만 남김 → 80번 포트 바인딩은 가능, 그 외 권한 모두 차단

Demo 4 — readOnlyRootFilesystem + emptyDir 쓰기

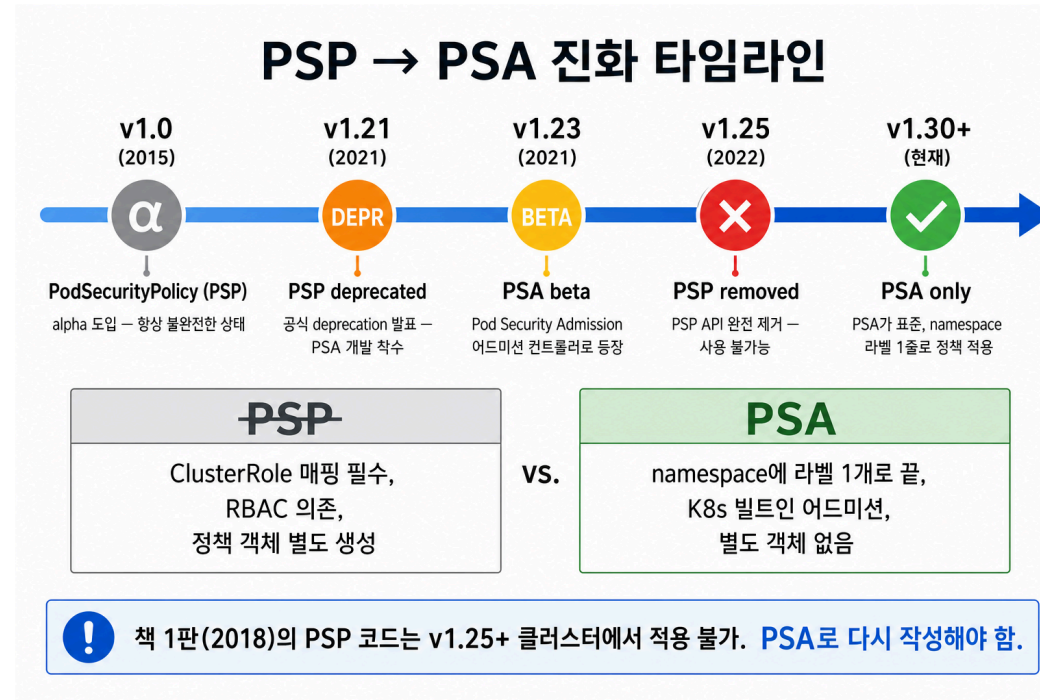
```
# security-context-readonly.yaml
spec:
  containers:
  - name: main
    image: busybox:stable
    command: ["sh","-c","sleep 3600"]
    securityContext:
      readOnlyRootFilesystem: true
    volumeMounts:
    - { name: tmp, mountPath: /tmp }
  volumes:
  - { name: tmp, emptyDir: {} }
```

```
$ kubectl exec readonly-pod -- sh -c 'echo hi > /etc/test'
sh: can't create /etc/test: Read-only file system

$ kubectl exec readonly-pod -- sh -c 'echo hello > /tmp/test && cat /tmp/test'
hello
```

- 컨테이너 루트(/etc , /usr , /bin)는 쓰기 불가 → 악성 바이너리 drop / ConfigMap 위변조 차단
- 쓰기 필요한 경로(/tmp , /var/cache)만 emptyDir 화이트리스트 → 재기동 시 깨끗한 상태로 복귀

13.3 PSP → PSA — 1판 책의 가장 큰 변경점



책 13.3절 PSP는 1.25에서 제거 → 1판 YAML 적용 불가. PSA는 namespace label 1줄로 동작.

PSA — 3 모드 × 3 프로필

Pod Security Admission — 3 모드 × 3 프로필

	privileged 제한 없음 (인프라용)	baseline 최소 보안 기준선	restricted 엄격한 보안 (권장)
 enforce	정책 위반 Pod 생성 거부 (admission denied)	정책 위반 Pod 생성 거부 (admission denied)	정책 위반 Pod 생성 거부 (admission denied)
 audit	감사 로그에 기록, Pod은 정상 생성	감사 로그에 기록, Pod은 정상 생성	감사 로그에 기록, Pod은 정상 생성
 warn	kubectl apply 시 경고 메시지 출력, Pod은 정상 생성	kubectl apply 시 경고 메시지 출력, Pod은 정상 생성	kubectl apply 시 경고 메시지 출력, Pod은 정상 생성

YAML (Namespace 예시)

```
apiVersion: v1
kind: Namespace
metadata:
  name: my-app
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
```

 namespace 라벨 1 개로 정책 적용 —
별도 ClusterRole/RoleBinding 불필요

 **권장 패턴:** 일반 ns는 **enforce=restricted**, kube-system은 **privileged**

권장 운영 흐름: 새 namespace는 warn=restricted → 며칠 관찰 → audit=restricted → 마지막 enforce=restricted. 처음부터 enforce를 켜면 기존 Pod 재시작 시 갑자기 거부됨.

Demo 5 — PSA restricted 어드미션 차단 (실측)

```
$ kubectl apply -f psa-restricted-namespace.yaml
namespace/psa-restricted created

$ kubectl get ns psa-restricted -o jsonpath='{.metadata.labels}'
{
  "pod-security.kubernetes.io/enforce": "restricted",
  "pod-security.kubernetes.io/audit":    "restricted",
  "pod-security.kubernetes.io/warn":     "restricted",
  "pod-security.kubernetes.io/enforce-version": "latest"
}
```

```
$ kubectl apply -n psa-restricted -f host-namespace-pod.yaml # hostNetwork=true
Error from server (Forbidden): pods "violator" is forbidden:
violates PodSecurity "restricted:latest":
- host namespaces (hostNetwork=true)
- allowPrivilegeEscalation != false (must set ... =false)
- unrestricted capabilities (must set ... drop=["ALL"])
- runAsNonRoot != true (must set ... runAsNonRoot=true)
- seccompProfile (must set ... type to "RuntimeDefault")
```

5가지 위반 사유가 한 번에 잡혀 API Server 어드미션 단계에서 즉시 거부 — kubelet까지 가지 못함.

Demo 5 (계속) — restricted를 통과하는 safe-pod

```
# safe-pod (restricted 통과 형태)
spec:
  containers:
  - name: main
    image: busybox:stable
    command: ["sleep","3600"]
    securityContext:
      runAsNonRoot: true
      runAsUser: 1000
      allowPrivilegeEscalation: false
      capabilities:
        drop: ["ALL"]
      seccompProfile:
        type: RuntimeDefault
```

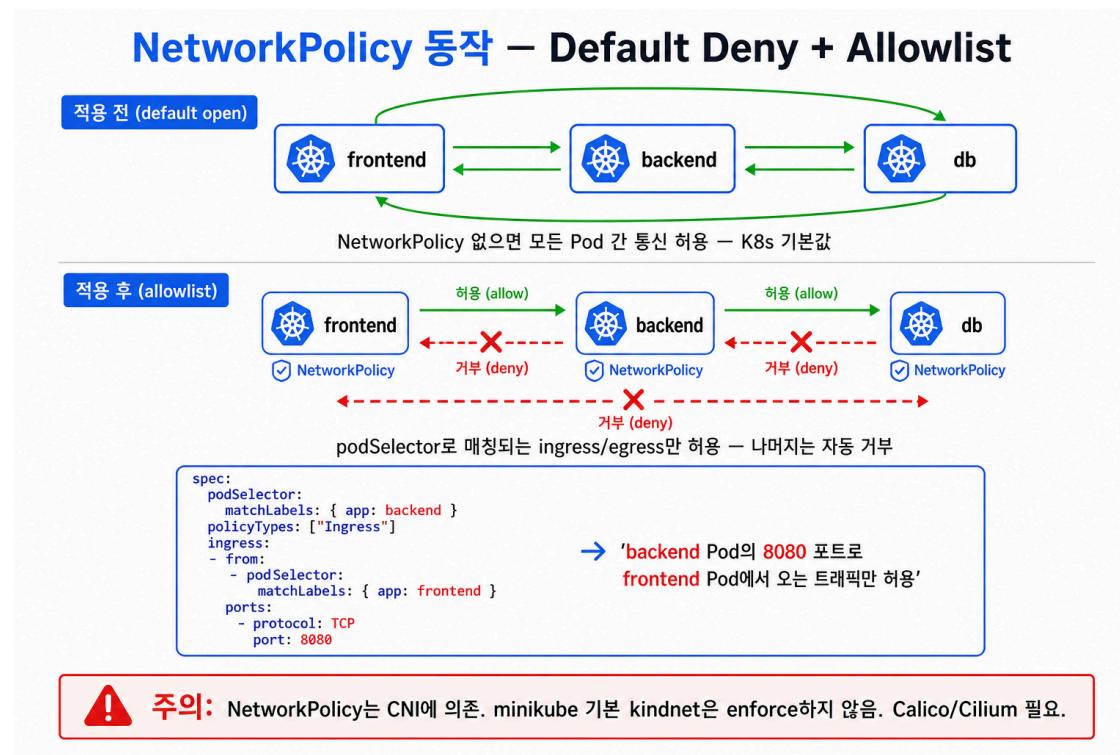
```
$ kubectl apply -n psa-restricted -f safe-pod.yaml
pod/safe-pod created
```

```
$ kubectl get pod -n psa-restricted
```

NAME	READY	STATUS	RESTARTS	AGE
safe-pod	1/1	Running	0	1s

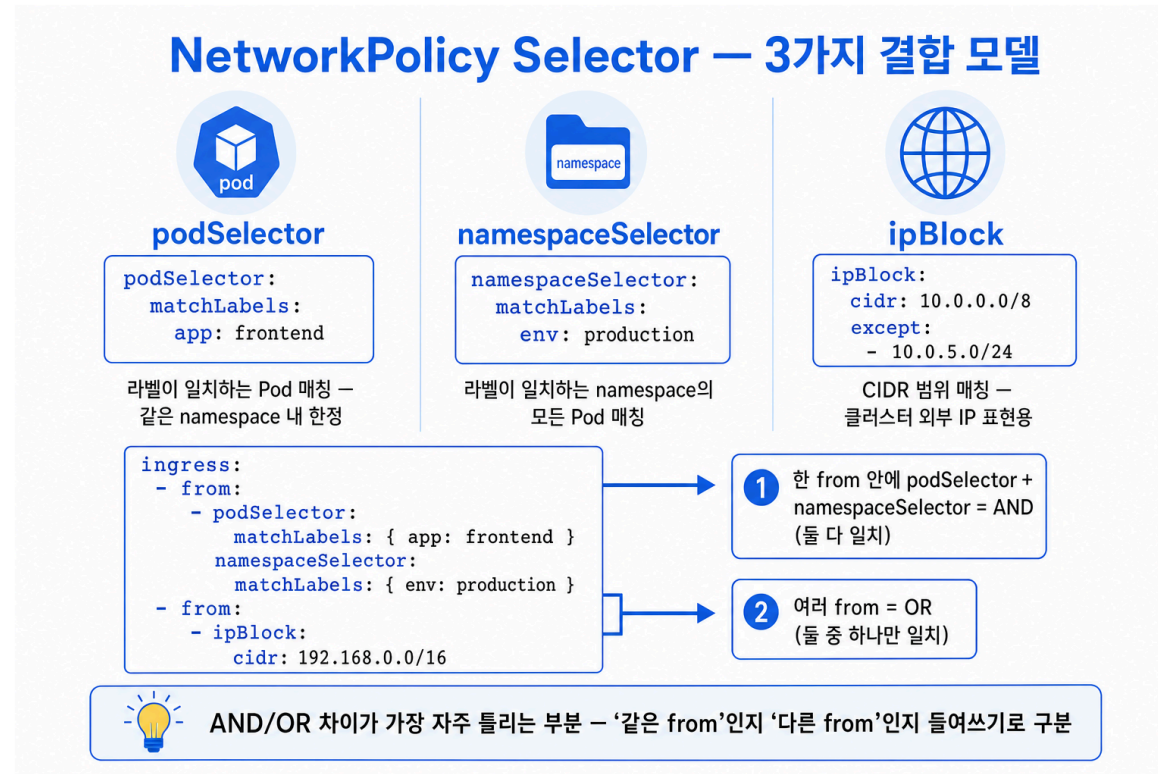
위 5개 필드 = PSA restricted 통과 최소 조건 = 13.2 Security Context 실무 기본값. Helm chart values에 박아두면 즉시 동작.

13.4 NetworkPolicy — Pod 간 통신 제어의 기본 모델



default open → default deny: 정책 없으면 모든 Pod 간 통신 허용. selector에 매칭된 Pod은 명시 룰만 허용. CNI 의존: NetworkPolicy는 API 객체일 뿐, 강제는 CNI가 함 — flannel은 무시. 오늘 데모는 Calico.

selector 3종 — podSelector / namespaceSelector / ipBlock



AND vs OR — YAML indent 한 단계 차이: 같은 `from` 항목 안에서 `podSelector + namespaceSelector` = AND. 별개 `from` 으로 `-` 분리 = OR. PR 리뷰에서 가장 흔한 NetworkPolicy 버그.

Demo 6 — Default Deny All Ingress (실측)

```
# networkpolicy-deny-all.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny-ingress, namespace: netpol-demo }
spec:
  podSelector: {}          # 빈 selector = 이 ns의 모든 Pod
  policyTypes: [ Ingress ]
  # ingress 룰 비어있음 = 모든 들어오는 트래픽 거부
```

```
# 정책 적용 전: frontend → backend
$ wget -q -T 3 -O - "http://$BACKEND_IP:8080"
hello from backend          # 200 OK

$ kubectl apply -f networkpolicy-deny-all.yaml

# 정책 적용 후 (같은 호출)
$ wget -q -T 3 -O - "http://$BACKEND_IP:8080"
wget: download timed out    # Calico 패킷 drop, RST 없이 timeout
```

운영 표준: prod namespace는 default-deny-ingress 먼저. "허용 통신만 명시"하는 화이트리스트 모델.

Demo 7 — Allow frontend → backend (실측)

```
# networkpolicy-allow-frontend.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-frontend-to-backend, namespace: netpol-demo }
spec:
  podSelector:
    matchLabels: { app: backend }           # backend Pod에 적용
  policyTypes: [ Ingress ]
  ingress:
    - from:
      - podSelector:
          matchLabels: { app: frontend }     # frontend만 허용
        ports:
          - { protocol: TCP, port: 8080 }
```

```
$ kubectl exec -n netpol-demo frontend -- wget -q -T 3 -O - "http://$BACKEND_IP:8080"
hello from backend                               # ← 허용 (200)

$ kubectl exec -n netpol-demo other -- wget -q -T 3 -O - "http://$BACKEND_IP:8080"
wget: download timed out                        # ← 거부 (other는 selector 미매칭)
```

합산 규칙: 같은 Pod에 매칭되는 정책의 ingress 룰은 OR(허용 합집합). deny-all + allow-frontend 조합이 깔끔하게 동작하는 이유.

Egress — Capital One 사고가 막혔을 한 줄

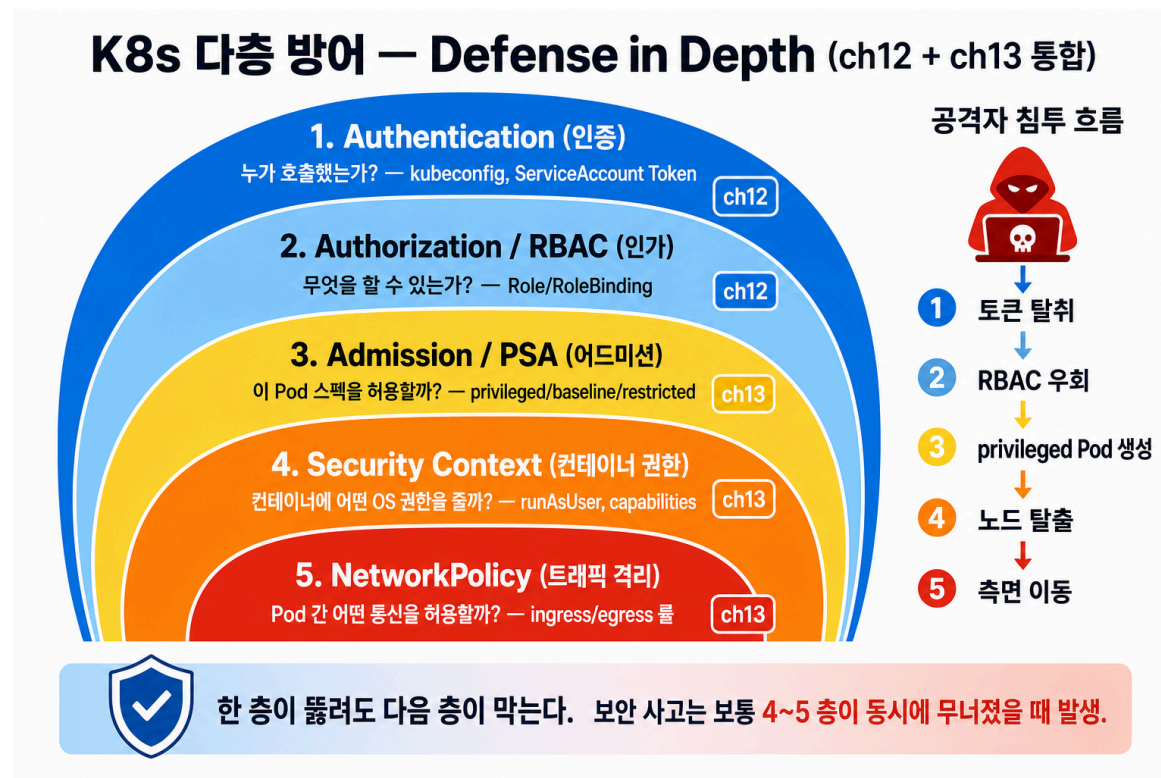
```
# 메타데이터 서버 차단 + 클러스터 내부만 허용
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: deny-imds, namespace: app }
spec:
  podSelector: {}
  policyTypes: [ Egress ]
  egress:
    - to:
      - ipBlock:
          cidr: 0.0.0.0/0
          except:
            - 169.254.169.254/32      # ← AWS/GCP/Azure IMDS 차단
            - 169.254.170.2/32       # ← ECS task metadata
    - to:
      - namespaceSelector: {}      # 클러스터 내부 ns 전체 허용
```

Egress 적용 시 주의 — DNS(53/UDP), API Server(443) 룰을 먼저 명시. 그 다음 외부 차단.

- DNS 빠뜨리면 모든 서비스 호출 깨짐 — `kube-system` ns의 53/UDP 명시 필수
- API Server(443) 빠지면 ServiceAccount 사용 불가

Capital One 사고: WAF Pod의 `169.254.169.254` egress 차단 단 한 줄로 SSRF → S3 1억 명 노출 흐름 전체 차단 가능.

Defense in Depth — 12장 + 13장 통합 결론



6계층 — L1 RBAC(12장), L2 PSA, L3 Security Context, L4 호스트 격리, L5 NetworkPolicy(13장), L6 런타임 감지(14장+). 하나만 뚫려도 막히도록 모두 켜다.

1판 vs 현행 K8s — 13장에서 바뀐 것

영역	책 1판 (2018, K8s 1.10)	현행 (2026, K8s 1.32)
Pod 보안 정책	PodSecurityPolicy(PSP)	PSA(1.25 GA), PSP 제거
보안 프로필	PSP YAML 직접 작성	<code>pod-security.kubernetes.io/*</code> namespace label
세밀 정책	PSP만	OPA Gatekeeper, Kyverno 추가
seccomp	annotation <code>seccomp.security.alpha.kubernetes.io/*</code>	<code>securityContext.seccompProfile</code> 정식 필드
AppArmor	annotation only	<code>securityContext.appArmorProfile</code> (1.30 GA)
NetworkPolicy	Ingress 중심	Egress 정식 지원 (1.8+), <code>endPort</code> (1.25), <code>namespaceSelector matchExpressions</code>
메모리 정책 표현	flannel/Calico 분기	AdminNetworkPolicy (1.30 alpha) — 클러스터 전역 우선순위 룰

오늘 데모에서 책과 다른 부분 — 책 13.3 PSP YAML → PSA `enforce=restricted` namespace label, 책 seccomp annotation → `seccompProfile: { type: RuntimeDefault }` 정식 필드. NET_BIND_SERVICE 예제는 현행과 동일.

운영 체크리스트 — 새 namespace 만들 때 반드시

```
# 1. namespace 생성 + PSA enforce=restricted (3 label)
kubectl create namespace ${NS}
kubectl label namespace ${NS} \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/audit=restricted \
  pod-security.kubernetes.io/warn=restricted

# 2. default-deny ingress + egress (DNS는 명시 허용)
kubectl apply -n ${NS} -f default-deny-with-dns.yaml

# 3. default ServiceAccount automount 차단
kubectl patch sa default -n ${NS} -p '{"automountServiceAccountToken":false}'

# 4. 감사 — privileged / hostNetwork Pod 검색
kubectl get pods -A -o json | jq -r '.items[] |
  select(.spec.hostNetwork==true or .spec.containers[].securityContext.privileged==true) |
  "\(.metadata.namespace)/\(.metadata.name)'"
```

위 4단계는 새 namespace 생성 직후 무조건 실행. GitOps에서 namespace 생성 시 위 4개를 함께 생성하는 Helm chart로 묶어 둬.

14장 예고 — Computing Resources

- 13장: 무엇을 할 수 있는가 (권한)
- 14장: 얼마나 쓸 수 있는가 (자원)

다룰 내용

- `requests` vs `limits` — 스케줄러와 OOMKiller가 보는 두 값
- QoS 클래스 (Guaranteed / Burstable / BestEffort) — Pod eviction 우선순위 결정
- LimitRange — namespace 단위 기본값 강제
- ResourceQuota — namespace 전체 자원 상한
- HPA(Horizontal Pod Autoscaler) — metrics-server 기반 자동 스케일링

권한 관리(13장) + 자원 관리(14장)가 합쳐져야 비로소 멀티팀 클러스터가 안전하게 운영됩니다.

Summary — 13장 한 화면 정리

4축으로 Pod의 권한을 닫는다

1. 호스트 namespace — `hostNetwork/hostPID/hostIPC = false` (시스템 Pod 외)
2. Security Context — `runAsNonRoot + drop ALL + readOnlyRootFilesystem + seccomp RuntimeDefault`
3. PSA — namespace label `enforce=restricted` 로 위 2개를 어드미션 단계에서 강제
4. NetworkPolicy — `default-deny + 명시 allow` (DNS/API egress는 명시)

오늘 실측한 것

- Demo 1: hostPID/Network로 노드 systemd, kubelet, NIC 모두 노출되는 모습
- Demo 2~4: `runAsUser=1000`, `drop ALL + NET_BIND_SERVICE`만, `readOnlyRootFilesystem`
- Demo 5: PSA `restricted`가 5가지 위반 사유를 한 번에 잡아 어드미션 거부 / `safe-pod`는 통과
- Demo 6: `deny-all` 적용 → `wget` timeout / Demo 7: `allow-frontend` → `frontend`만 통과, `other`는 timeout

기억할 단 한 줄

RBAC만으로 끝나지 않는다. PSA + Security Context + NetworkPolicy의 3중 격벽이 있어야 침해의 영향을 한 컨테이너 안에 가둘 수 있다.

질문 & 토론

자주 나오는 질문 3가지

- Q. PSA만 있으면 OPA/Kyverno 없어도 되나?

A. 표준 워크로드는 PSA로 충분. 커스텀 룰(이미지 레지스트리 화이트리스트, 라벨 강제 등)이 필요하면 Kyverno 추가.

- Q. NetworkPolicy를 만들었는데 효과가 없는데?

A. CNI 확인 필수. flannel은 NetworkPolicy 무시 → Calico/Cilium으로 교체.

- Q. default ServiceAccount의 token은 자동 마운트되는데?

A. SA에 `automountServiceAccountToken: false`, 또는 Pod spec에서 명시적으로 false. 1.24부터는 더 이상 Secret으로 영구 토큰을 만들지 않고 projected token만 사용 (12장에서 다룬 것과 연결).